

# Unix & Shell Programming

## Module 8 – Shell Programming

### Question & Answer Notes

BCA Semester 6 · MAKAUT

#### Q1. What is a shell? Explain different types of shells.

A **shell** is a program that acts as an interface between the **user** and the **UNIX kernel**. It reads commands typed by the user, interprets them, and passes them to the kernel for execution. It also displays the output back to the user.

Think of it this way: the kernel is the core of UNIX, but you cannot talk to it directly. The shell is the “translator” in between.

##### What a shell does:

- Reads and executes commands.
- Supports variables, loops, and conditions (scripting).
- Manages input/output redirection and pipes.
- Provides a programming environment (shell scripts).

##### Types of Shells:

| Shell | Full Name          | Command   | Notes                             |
|-------|--------------------|-----------|-----------------------------------|
| sh    | Bourne Shell       | /bin/sh   | Original UNIX shell, very basic   |
| bash  | Bourne Again Shell | /bin/bash | Most popular; default in Linux    |
| csch  | C Shell            | /bin/csh  | C-like syntax; good for scripting |
| tcsh  | Tenex C Shell      | /bin/tcsh | Improved version of csh           |
| ksh   | Korn Shell         | /bin/ksh  | Combines sh and csh features      |
| zsh   | Z Shell            | /bin/zsh  | Feature-rich; default on macOS    |

**bash** is the most widely used shell. When we write shell scripts, we usually write them for bash. The first line of a bash script is `#!/bin/bash` (called the shebang line).

#### Q2. Explain the uses of shell programming.

**Shell programming** (shell scripting) means writing a series of shell commands in a file (called a **shell script**) so they can be executed together automatically.

##### A simple shell script example:

```
#!/bin/bash
echo "Hello, World!"
echo "Today is: $(date)"
```

##### Uses of shell programming:

1. **Automation** – Automate repetitive tasks like taking backups, renaming files, or generating reports. Instead of typing the same commands every day, run a script.
2. **System Administration** – Automate system tasks like user creation, disk monitoring, log cleanup, and software installation.
3. **Batch Processing** – Process large numbers of files at once. For example, convert all .txt files in a folder to .csv.
4. **Scheduling Jobs** – Shell scripts are used with `cron` to schedule tasks to run at specific times (e.g., backup every night at 2 AM).
5. **Custom Commands** – Create shortcuts for long or complex commands that you use frequently.
6. **File Management** – Automate copying, moving, searching, and organizing files.
7. **Monitoring** – Write scripts to monitor CPU usage, memory, disk space, and alert administrators when thresholds are crossed.
8. **Quick Prototyping** – Test logic or workflows quickly without writing a full program in C or Python.

#### How to run a shell script:

```
chmod +x myscript.sh      # give execute permission
./myscript.sh             # run the script
bash myscript.sh          # alternative way to run
```

### Q3. Explain system variables and user-defined variables.

**Variables** in shell are used to store values (text or numbers) that can be used later in the script.

#### A. System Variables (Environment Variables)

These are **pre-defined** variables set by the shell or the operating system. They are written in **UPPERCASE** by convention.

| Variable | Meaning                                    |
|----------|--------------------------------------------|
| HOME     | Home directory of the current user         |
| USER     | Name of the logged-in user                 |
| PATH     | Directories where shell looks for commands |
| PWD      | Current working directory                  |
| SHELL    | Path of the current shell                  |
| PS1      | Primary command prompt string              |
| TERM     | Type of terminal being used                |
| LANG     | Language/locale setting                    |

#### Example:

```
echo $HOME      # prints /home/student
echo $USER      # prints student
echo $PATH      # prints the PATH variable
echo $PWD       # prints current directory
```

## B. User-Defined Variables

These are variables that **you create** in your script to store data.

### Rules for naming:

- Can contain letters, digits, and underscore (\_).
- Must start with a letter or underscore (not a digit).
- No spaces around the = sign when assigning.
- Case-sensitive: **Name** and **name** are different variables.

### Example:

```
#!/bin/bash
name="Ravi"           # assign a string value
age=21                # assign a number
city="Kolkata"

echo "Name: $name"
echo "Age : $age"
echo "City: $city"

# Unset (delete) a variable
unset city
echo "City: $city"    # prints empty
```

When assigning: `name="Ravi"` (no spaces). When using: `$name` (dollar sign before the name).

## Q4. Explain the use of single quotes, double quotes, and backslash.

Quotes and backslash control how the shell interprets special characters.

### 1. Double Quotes " "

- Most special characters lose their meaning inside double quotes.
- However, \$, ` (backtick), and \ are still interpreted.
- Variables are **expanded** (substituted) inside double quotes.

```
name="Ravi"
echo "Hello $name"      # Output: Hello Ravi
echo "2 * 3 = ?"        # Output: 2 * 3 = ?    (* not expanded)
echo "Home: $HOME"      # Output: Home: /home/student
```

### 2. Single Quotes ' '

- **All** special characters lose their meaning inside single quotes.
- Variables are **NOT** expanded. Everything is treated as literal text.

```
name="Ravi"
echo 'Hello $name'      # Output: Hello $name    (NOT expanded)
echo 'Today is $(date)' # Output: Today is $(date)
```

```
echo '2 * 3'           # Output: 2 * 3
```

### 3. Backslash \ (Escape Character)

- The backslash **escapes** the very next character, making it literal.
- Used to include special characters like \$, ", \, etc. in output.

```
echo "Price: \$100"      # Output: Price: $100
echo "Say \"Hello\""     # Output: Say "Hello"
echo "Line1\nLine2"      # \n treated as newline with echo -e
echo "Tab\there"         # \t is tab
```

| Quote Type  | Variable Expansion?        | Special Chars?        |
|-------------|----------------------------|-----------------------|
| Double " "  | Yes (\$var expands)        | Most are disabled     |
| Single ' '  | No (everything is literal) | All are disabled      |
| Backslash \ | Escapes next char only     | One char made literal |

## Q5. Explain command substitution with examples.

**Command substitution** allows you to use the **output of a command** as a value – for example, to assign it to a variable or include it inside a string.

**Two ways to do command substitution:**

1. Using backticks: `command`
2. Using \$(...): \$(command) (recommended – can be nested)

**Examples:**

```
#!/bin/bash

# Store today's date in a variable
today=$(date)
echo "Today is: $today"

# Store current directory
dir=$(pwd)
echo "You are in: $dir"

# Count number of files in current directory
count=$(ls | wc -l)
echo "Number of files: $count"

# Use inside a string directly
echo "Logged in as: $(whoami)"

# Backtick style (older method)
hostname=`hostname`
echo "Machine name: $hostname"

# Nested substitution (only works with $(...))
```

```
echo "Files in home: $(ls $(echo $HOME) | wc -l)"
```

**Output:**

```
Today is: Sat Jun 06 10:30:00 IST 2026
You are in: /home/student
Number of files: 12
Logged in as: student
Machine name: mypc
Files in home: 25
```

Prefer `$(...)` over backticks because it is easier to read and supports nesting.

## Q6. Explain the use of `let` for evaluating expressions.

By default, the shell treats all variable values as **strings**. To do **arithmetic**, we need special commands. `let` is one such built-in command used to evaluate integer arithmetic expressions.

**Syntax:** `let expression`

**Examples:**

```
#!/bin/bash

# Basic arithmetic using let
a=10
b=3

let sum=a+b
let diff=a-b
let prod=a*b
let quot=a/b
let rem=a%b

echo "Sum          = $sum"      # 13
echo "Difference  = $diff"     # 7
echo "Product     = $prod"     # 30
echo "Quotient    = $quot"     # 3
echo "Remainder   = $rem"      # 1

# Increment / Decrement
let a++
echo "a after a++ = $a"        # 11

let a--
echo "a after a-- = $a"        # 10

# Power (exponentiation)
let power=2**8
echo "2 to the power 8 = $power" # 256
```

**Other ways to do arithmetic in shell:**

```
# Using $(( )) -- most common and recommended
result=$((10 + 5))
echo $result           # 15

x=7
echo $((x * x))        # 49

# Using expr command (older method)
result=$(expr 10 + 5)
echo $result           # 15
```

| Method  | Example                   |
|---------|---------------------------|
| let     | let result=a+b            |
| \$(( )) | result=\$((a+b))          |
| expr    | result=\$(expr \$a + \$b) |

**Q7. Write a script to take input from the user.**

The read command is used to take input from the user in shell scripting.

**Syntax:** `read [options] variable_name`

**Script 1 – Basic input:**

```
#!/bin/bash
echo "Enter your name:"
read name
echo "Hello, $name! Welcome."
```

**Script 2 – Input with prompt on same line (-p):**

```
#!/bin/bash
read -p "Enter your name: " name
read -p "Enter your age : " age
echo "Name: $name, Age: $age"
```

**Script 3 – Silent input for password (-s):**

```
#!/bin/bash
read -p "Username: " uname
read -sp "Password: " pass
echo ""
echo "Login attempted for: $uname"
```

**Script 4 – Read multiple values at once:**

```
#!/bin/bash
echo "Enter three numbers:"
read a b c
```

```
echo "You entered: $a, $b, $c"
sum=$((a + b + c))
echo "Sum = $sum"
```

### Script 5 – Simple calculator:

```
#!/bin/bash
read -p "Enter first number : " n1
read -p "Enter second number: " n2
echo "Sum          = $((n1 + n2))"
echo "Difference = $((n1 - n2))"
echo "Product    = $((n1 * n2))"
echo "Quotient    = $((n1 / n2))"
```

| Option   | Meaning                                       |
|----------|-----------------------------------------------|
| -p "msg" | Show prompt message on same line              |
| -s       | Silent mode (input not shown – for passwords) |
| -n N     | Read only N characters                        |
| -t N     | Timeout after N seconds                       |

## Q8. Explain command-line parameters (\$1, \$2, etc.).

When you run a shell script, you can pass values to it directly on the command line. These are called **command-line arguments** or **positional parameters**.

```
./script.sh arg1 arg2 arg3
```

Inside the script, these arguments are accessed using:

| Parameter | Meaning                           |
|-----------|-----------------------------------|
| \$0       | Name of the script itself         |
| \$1       | First argument                    |
| \$2       | Second argument                   |
| \$3       | Third argument (and so on...)     |
| \$#       | Total number of arguments passed  |
| \$*       | All arguments as a single string  |
| @         | All arguments as separate strings |
| ?         | Exit status of last command       |
| \$\$      | PID of the current script         |

### Example script (greet.sh):

```
#!/bin/bash
echo "Script name : $0"
echo "First arg   : $1"
echo "Second arg  : $2"
echo "Total args   : $# "
echo "All args     : $*"
```

**Run it:**

```
./greet.sh Ravi 21
```

**Output:**

```
Script name : ./greet.sh
First arg   : Ravi
Second arg  : 21
Total args  : 2
All args    : Ravi 21
```

**Another example – add two numbers:**

```
#!/bin/bash
# Usage: ./add.sh 10 20
echo "Sum = $(( $1 + $2 ))"
```

If you pass more than 9 arguments, use curly braces for 10th onward: \${10}, \${11}, etc.

## Q9. Explain the use of if statements in shell.

**if** statements are used for **decision making** in shell scripts. They test a condition and execute a block of code based on whether it is true or false.

**Basic Syntax:**

```
if [ condition ]
then
    # commands if true
fi
```

**if-else Syntax:**

```
if [ condition ]
then
    # commands if true
else
    # commands if false
fi
```

**if-elif-else Syntax:**

```
if [ condition1 ]
then
    commands
elif [ condition2 ]
then
    commands
else
    commands
```



```
fi
```

**Common test operators:**

| Type    | Operator | Meaning               |
|---------|----------|-----------------------|
| Numeric | -eq      | Equal to              |
|         | -ne      | Not equal             |
|         | -gt      | Greater than          |
|         | -lt      | Less than             |
|         | -ge      | Greater than or equal |
|         | -le      | Less than or equal    |
| String  | =        | Strings are equal     |
|         | !=       | Strings are not equal |
|         | -z       | String is empty       |
|         | -n       | String is not empty   |
| File    | -f       | Is a regular file     |
|         | -d       | Is a directory        |
|         | -e       | File/dir exists       |
|         | -r       | File is readable      |

**Example 1 – Pass/Fail:**

```
#!/bin/bash
read -p "Enter marks: " marks
if [ $marks -ge 40 ]
then
    echo "Result: PASS"
else
    echo "Result: FAIL"
fi
```

**Example 2 – Grade system:**

```
#!/bin/bash
read -p "Enter marks: " m
if [ $m -ge 80 ]
then
    echo "Grade: A"
elif [ $m -ge 60 ]
then
    echo "Grade: B"
elif [ $m -ge 40 ]
then
    echo "Grade: C"
else
    echo "Grade: F"
fi
```

## Q10. Explain for, while, and until loops in shell scripting.

Loops allow a block of commands to be repeated multiple times.

### A. for Loop

Used when you know how many times to repeat, or to iterate over a list.

#### Syntax:

```
for variable in list
do
    commands
done
```

#### Examples:

```
#!/bin/bash
# Print numbers 1 to 5
for i in 1 2 3 4 5
do
    echo "Number: $i"
done

# Using a range
for i in {1..5}
do
    echo "Count: $i"
done

# Loop over files
for file in *.txt
do
    echo "File found: $file"
done

# C-style for loop
for (( i=1; i<=5; i++ ))
do
    echo "i = $i"
done
```

### B. while Loop

Repeats as long as a condition is **true**.

#### Syntax:

```
while [ condition ]
do
    commands
done
```

**Examples:**

```
#!/bin/bash
# Print 1 to 5 using while
i=1
while [ $i -le 5 ]
do
    echo "i = $i"
    let i++
done

# Read lines from a file
while read line
do
    echo "Line: $line"
done < myfile.txt
```

**C. until Loop**

Repeats as long as a condition is **false** (opposite of while).

**Syntax:**

```
until [ condition ]
do
    commands
done
```

**Example:**

```
#!/bin/bash
# Print 1 to 5 using until
i=1
until [ $i -gt 5 ]
do
    echo "i = $i"
    let i++
done
```

| Loop  | Runs when                 | Use case                       |
|-------|---------------------------|--------------------------------|
| for   | For each item in a list   | Known number of iterations     |
| while | Condition is <b>true</b>  | Repeat until something changes |
| until | Condition is <b>false</b> | Opposite logic of while        |

**Loop control:**

```
break      # exit the loop immediately
continue   # skip current iteration and go to next
```

## Q11. How to check exit status in shell scripting?

Every command in UNIX returns an **exit status** (also called **return code**) when it finishes:

- **0** means the command ran **successfully**.
- **Non-zero** (1, 2, 127, etc.) means the command **failed** or there was an error.

The special variable `$?` holds the exit status of the last executed command.

**Example 1 – Basic exit status check:**

```
#!/bin/bash
ls /home
echo "Exit status of ls: $?"      # 0 if success

ls /nonexistent_folder
echo "Exit status of ls: $?"      # non-zero (error)
```

**Example 2 – Check if a command succeeded:**

```
#!/bin/bash
cp file.txt /backup/

if [ $? -eq 0 ]
then
    echo "File copied successfully."
else
    echo "Error: Copy failed!"
fi
```

**Example 3 – Using exit status with grep:**

```
#!/bin/bash
grep "root" /etc/passwd
if [ $? -eq 0 ]
then
    echo "User root found."
else
    echo "User root not found."
fi
```

**Example 4 – Setting your own exit status with exit:**

```
#!/bin/bash
read -p "Enter age: " age
if [ $age -lt 0 ]
then
    echo "Invalid age!"
    exit 1      # exit with error code 1
fi
echo "Age is: $age"
exit 0         # exit with success
```

**Common exit status codes:**

| Exit Code | Meaning                          |
|-----------|----------------------------------|
| 0         | Success                          |
| 1         | General error                    |
| 2         | Misuse of shell command          |
| 126       | Command found but not executable |
| 127       | Command not found                |
| 130       | Script terminated by Ctrl+C      |

Always check `$?` right after the command you want to test, because every new command overwrites `$?` with its own exit status.

---

End of Module 8 Notes